

Universita' degli Studi
Corso di Laurea in Ingegneria Informatica

Introduzione al Trattamento Quantistico delle Informazioni

PROGETTO DI LABORATORIO

AREA HADAMARD GATE

Implementazione VHDL di un gate di Hadamard 2x2
su FPGA Boolean Board (Spartan-7)

Indice del documento:

Sezione 1 - Listato di codice e spiegazione dettagliata

Sezione 2 - Risultato del Linter

Sezione 3 - Risultati di implementazione (risorse, F_max, potenza)

Sezione 4 - Risultati di test-bench behavioral (4 input)

Studente: **Raffaele Della Porta**

Matricola: **0322500029**

Docente: **Sergio Spano'**

Anno Accademico: **2025 / 2026**

Progetto di Laboratorio

AREA HADAMARD GATE

Implementazione VHDL di un gate di Hadamard 2x2
su FPGA Boolean Board (Spartan-7)

Corso: **Introduzione al Trattamento Quantistico delle Informazioni**
Sezione 1 del Report: **Listato di codice e spiegazione dettagliata**

Documento generato automaticamente con ReportLab

1. Sezione 1 — Listato di codice e spiegazione dettagliata

1.1 Inquadramento teorico

Il gate di Hadamard è la porta quantistica a 1-qubit più importante nei circuiti di Deutsch–Jozsa, Grover e Shor: trasforma uno stato di base in una sovrapposizione bilanciata. La sua matrice è:

$$H = (1/\sqrt{2}) * \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Applicata al vettore di stato $|\psi\rangle = [a; b]^T$ produce:

$$H * |\psi\rangle = (1/\sqrt{2}) * \begin{bmatrix} a+b \\ a-b \end{bmatrix}$$

L'obiettivo del progetto è realizzare questa trasformazione in hardware su FPGA, usando aritmetica a virgola fissa a **12 bit con segno** e **8 bit di parte frazionaria** (formato **Q3.8**: 1 bit di segno, 3 bit interi, 8 bit frazionari). Il range rappresentabile è $[-8.000, +7.996]$, con risoluzione $1/256 \approx 3.9 \cdot 10^{-3}$.

La costante $1/\sqrt{2} \approx 0.70710678$ in Q3.8 vale:

$$\text{round}(0.70710678 * 2^8) = 181 = 0000_1011_0101 \text{ (base 2)}$$

1.2 Architettura del progetto

La specifica richiede che **tutti gli ingressi e le uscite della top-level entity siano collegati a registri**. Sono quindi presenti due banchi di registri: uno in ingresso (`a_reg`, `b_reg`) e uno in uscita (`y0_reg`, `y1_reg`). Il calcolo combinatorio è incapsulato nel modulo **hadamard_core**, riusabile e parametrico. I valori della matrice (1, 1, 1, -1, costante $1/\sqrt{2}$) non sono input ma **costanti**, come permesso dal SUGGERIMENTO della traccia.

```
A_CONST --> [ REG a_reg (12b) ] --\
                                     +--> [ hadamard_core ] --> [ REG y0_reg ] --> y0_obs[5:0]
B_CONST --> [ REG b_reg (12b) ] --/                                     |
                                     |                                     \n

hadamard_core:
  sum  = a + b
  diff = a - b
  prod = (...) * 181
  y    = prod >> 8
```

Schema a blocchi della top-level entity con registri I/O.

1.3 Struttura dei file sorgente

File	Ruolo
src/hadamard_core.vhd	Modulo combinatorio: implementa $H \cdot v$
src/hadamard_top.vhd	Top-level: registri I/O, costanti A,B, istanza core
sim/tb_hadamard.vhd	Testbench behavioral con 4 stimoli
constraints/hadamard.xdc	Pin assignment per la Boolean Board

1.4 Listato — hadamard_core.vhd

```
-----
-- File      : hadamard_core.vhd
-- Descrizione : Logica combinatoria del gate di Hadamard 2x2.
--            Calcola  $H \cdot [a;b] = (1/\sqrt{2}) \cdot [a+b ; a-b]$ .
--            Aritmetica a virgola fissa Q3.8 (12 bit signed, 8 frazionari).
-- Target    : Boolean Board (Xilinx Spartan-7 XC7S50)
-----

library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

entity hadamard_core is
  generic (
    data_width : integer := 12; -- larghezza totale del dato
    frac_bits  : integer := 8  -- bit di parte frazionaria
  );
  port (
    a_in  : in    signed(data_width - 1 downto 0); -- ampiezza qubit |0>
    b_in  : in    signed(data_width - 1 downto 0); -- ampiezza qubit |1>
    y0_out : out  signed(data_width - 1 downto 0); -- (a+b)/sqrt(2)
    y1_out : out  signed(data_width - 1 downto 0); -- (a-b)/sqrt(2)
  );
end entity hadamard_core;

architecture rtl of hadamard_core is

  -- 1/sqrt(2) in Q3.8: round(0.70710678 * 256) = 181 = "00 0010110101"
  -- Lo rappresentiamo su 12 bit signed (valore positivo).
  constant inv_sqrt2 : signed(data_width - 1 downto 0) :=
    to_signed(181, data_width);

  -- Somma e differenza estese di 1 bit per evitare overflow.
  signal sum_ab  : signed(data_width downto 0);
  signal diff_ab : signed(data_width downto 0);

  -- Prodotti completi: (DATA_WIDTH+1) * DATA_WIDTH = 2*DATA_WIDTH+1 bit
  signal prod_sum  : signed(2 * data_width downto 0);
  signal prod_diff : signed(2 * data_width downto 0);

begin

  -- Somma e differenza con estensione di segno.
  sum_ab <= resize(a_in, data_width + 1) + resize(b_in, data_width + 1);
  diff_ab <= resize(a_in, data_width + 1) - resize(b_in, data_width + 1);

  -- Moltiplicazione per 1/sqrt(2) (Q3.8 * Q3.8 -> Q6.16 esteso).
  prod_sum <= sum_ab * inv_sqrt2;
  prod_diff <= diff_ab * inv_sqrt2;

  -- Riallineamento Q -> Q3.8: scarto FRAC_BITS bit bassi e prendo
  -- DATA_WIDTH bit (con saturazione implicita affidata al range dei segnali).
  -- prod ha 2*DATA_WIDTH+1 bit con 2*FRAC_BITS bit frazionari;
  -- per tornare a FRAC_BITS frazionari devo shiftare a destra di FRAC_BITS.
  y0_out <= resize(shift_right(prod_sum,  frac_bits), data_width);
  y1_out <= resize(shift_right(prod_diff, frac_bits), data_width);

end architecture rtl;
```

Spiegazione dettagliata

Elemento	Cosa fa	Perché è così
library IEEE; use NUMERIC_STD;	Importa i tipi signed/unsigned e operatori aritmetici IEEE.	Standard IEEE consigliato; evita ambiguità di std_logic_arith.
generic DATA_WIDTH, FRAC_BITS	Parametrizza la larghezza del dato e i bit frazionari.	Per il riutilizzo del modulo per altri formati Q senza riscrivere.

a_in, b_in : signed	Ingressi a 12 bit con segno (Q3.8).	I dati sono in [-8, +8) con risoluzione 1/256.
INV_SQRT2 := to_signed(181, 12)	Costante $1/\sqrt{2}$ in Q3.8.	$\text{round}(0.70710678 \cdot 256) = 181$; errore di quantizzazione $\approx 1.3 \cdot 10^{-4}$.
sum_ab, diff_ab : signed(12)	Operazioni a + b e a - b su 13 bit.	Estensione di 1 bit per evitare overflow quando $ a + b \geq 8$.
prod_sum, prod_diff : signed(24)	Prodotti completi a 25 bit.	$\text{signed}(13) \cdot \text{signed}(12) = \text{signed}(25)$; nessuna perdita di precisione.
shift_right(prod, FRAC_BITS)	Divide per 2^8 , ovvero riallinea il risultato a Q3.8.	signed, shift_right è aritmetico → preserva il segno.
resize(..., DATA_WIDTH)	Ritorna a 12 bit.	I bit alti sono replica del segno per gli ingressi normalizzati.

Aspetto numerico chiave. Il prodotto $\text{signed}(13b) \times \text{signed}(12b)$ produce 25 bit con 16 bit frazionari. Dopo `shift_right(...,8)` restano 17 bit con 8 frazionari; `resize` ne tiene 12. Vivado, per questa moltiplicazione, inferisce un blocco **DSP48E1** per ciascun prodotto (2 totali).

1.5 Listato — hadamard_top.vhd

```
-----
-- File : hadamard_top.vhd
-----

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity hadamard_top is
generic(
    data_width : integer := 12;
    frac_bits   : integer := 8
);
port(
    clk : in std_logic;
    rst : in std_logic;

    -- solo pin realmente disponibili
    y0_obs : out std_logic_vector(5 downto 0);
    y1_obs : out std_logic_vector(5 downto 0)
);
end entity;

architecture rtl of hadamard_top is

    constant a_const : signed(data_width-1 downto 0)
        := to_signed(256,data_width);

    constant b_const : signed(data_width-1 downto 0)
        := to_signed(0,data_width);

    signal a_reg,b_reg : signed(data_width-1 downto 0);

    signal y0_c,y1_c : signed(data_width-1 downto 0);

    signal y0_reg,y1_reg : signed(data_width-1 downto 0);

    component hadamard_core
    generic(
        data_width : integer := 12;
        frac_bits   : integer := 8
    );
    port(
        a_in   : in signed(data_width-1 downto 0);
        b_in   : in signed(data_width-1 downto 0);

        y0_out : out signed(data_width-1 downto 0);
        y1_out : out signed(data_width-1 downto 0)
    );
    end component;

begin

    -----
    -- input registers
    -----

    p_in_reg : process(clk)
    begin

        if rising_edge(clk) then

            if rst='1' then
                a_reg<=(others=>'0');
                b_reg<=(others=>'0');
```

```

        else
            a_reg<=a_const;
            b_reg<=b_const;
        end if;

    end if;

end process;

-----
-- core
-----

u_core : hadamard_core
generic map(
    data_width => data_width,
    frac_bits  => frac_bits
)
port map(
    a_in  => a_reg,
    b_in  => b_reg,
    y0_out => y0_c,
    y1_out => y1_c
);

-----
-- output registers
-----

p_out_reg : process(clk)
begin

    if rising_edge(clk) then

        if rst='1' then
            y0_reg<=(others=>'0');
            y1_reg<=(others=>'0');
        else
            y0_reg<=y0_c;
            y1_reg<=y1_c;
        end if;

    end if;

end process;

-----
-- esporta solo bit realmente collegati
-----

y0_obs(0) <= y0_reg(0);
y0_obs(1) <= y0_reg(1);
y0_obs(2) <= y0_reg(4);
y0_obs(3) <= y0_reg(6);
y0_obs(4) <= y0_reg(7);
y0_obs(5) <= y0_reg(11);

y1_obs(0) <= y1_reg(1);
y1_obs(1) <= y1_reg(2);
y1_obs(2) <= y1_reg(3);
y1_obs(3) <= y1_reg(4);
y1_obs(4) <= y1_reg(6);
y1_obs(5) <= y1_reg(7);

end architecture;

```

Spiegazione dettagliata

1. Porte fisiche — clk e rst sono gli unici input osservabili sulla board; il vettore di stato è interno (costante), come consentito dalla traccia. Le uscite top-level sono due bus da 6 bit (*y0_obs*, *y1_obs*) che esportano un sottoinsieme di bit dei registri di uscita, coerentemente con i pin realmente disponibili nel file .xdc. Internamente i segnali di dato restano comunque a 12 bit Q3.8.

2. Costanti del vettore stato — $A_CONST = 256$, $B_CONST = 0 \rightarrow$ in Q3.8 valgono +1.0 e 0.0, cioè lo stato |0■. Cambiando solo queste due righe si testano in hardware tutti gli stati (|1■, |+■, |-■, ...).

3. Registri di ingresso p_in_reg — processo sensibile al solo clk: inferisce flip-flop sincroni con reset sincrono attivo alto. Forzando i registri all'inizio del clock domain, blindiamo il timing del core.

4. Istanza u_core — mappatura diretta dei registri sui port del core. I generic vengono propagati, così il top resta scalabile.

5. Registri di uscita p_out_reg — stesso schema: catturano *y0_c/y1_c* al fronte di salita. Architetturealmente il core resta quindi incapsulato tra uno stadio di registri di ingresso e uno di uscita.

6. Driver delle uscite — il top non esporta l'intero vettore a 12 bit, ma solo alcuni bit significativi dei registri *y0_reg* e *y1_reg* verso i pin realmente cablati sulla board. Questa scelta non altera la larghezza interna dei dati, che rimane 12 bit signed Q3.8 come richiesto.

1.6 Listato — tb_hadamard.vhd (testbench behavioral)

```
-----
-- File      : tb_hadamard.vhd
-- Descrizione : Testbench BEHAVIORAL del gate di Hadamard 2x2.
--            Verifica 4 tipologie di input:
--            1) Stato |0> = [1.0 ; 0.0]      -> attesa [+0.707 ; +0.707]
--            2) Stato |1> = [0.0 ; 1.0]      -> attesa [+0.707 ; -0.707]
--            3) Stato |+> = [0.707 ; 0.707]  -> attesa [+1.0   ; 0.0   ]
--            4) Stato |-> = [0.707 ; -0.707] -> attesa [ 0.0   ; +1.0   ]
-----

library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;

entity tb_hadamard is
end entity tb_hadamard;

architecture sim of tb_hadamard is

  constant data_width : integer := 12;
  constant frac_bits  : integer := 8;
  constant clk_period : time    := 10 ns;  -- 100 MHz

  -- DUT: istanzio direttamente il core per poter cambiare gli ingressi.
  component hadamard_core is
    generic (
      data_width : integer := 12;
      frac_bits  : integer := 8
    );
    port (
      a_in  : in    signed(DATA_WIDTH - 1 downto 0);
      b_in  : in    signed(DATA_WIDTH - 1 downto 0);
      y0_out : out  signed(DATA_WIDTH - 1 downto 0);
      y1_out : out  signed(DATA_WIDTH - 1 downto 0)
    );
  end component hadamard_core;

  signal a_in, b_in : signed(data_width - 1 downto 0) := (others => '0');
  signal y0, y1    : signed(data_width - 1 downto 0);

  -- Funzione di utilita': converte un real (numero in virgola mobile)
  -- nella sua rappresentazione Q3.8 a 12 bit signed.

  function to_q (
    val : real
  ) return signed is
  begin
    return to_signed(integer(val * real(2 ** FRAC_BITS)), DATA_WIDTH);
  end function to_q;

  -- Procedura di check con tolleranza (LSB).

  procedure check (
    constant name : string;
    signal  got    : signed;
    constant exp  : real
  ) is
    variable exp_q : signed(DATA_WIDTH - 1 downto 0);
    variable diff  : integer;

  begin
    exp_q := to_q(exp);
```

```

diff := to_integer(got) - to_integer(exp_q);

if (diff < 0) then
    diff := -diff;
end if;

if (diff <= 2) then
    report name & " OK (got=" & integer'image(to_integer(got)) &
        " exp=" & integer'image(to_integer(exp_q)) & ")"
        severity note;
else
    report name & " FAIL (got=" & integer'image(to_integer(got)) &
        " exp=" & integer'image(to_integer(exp_q)) & ")"
        severity error;
end if;

end procedure check;

begin

-----
-- Istanza DUT
-----
dut : component hadamard_core
generic map (
    data_width => DATA_WIDTH, frac_bits => FRAC_BITS
)
port map (
    a_in  => a_in,
    b_in  => b_in,
    y0_out => y0,
    y1_out => y1
);

-----
-- Stimoli
-----
stim_proc : process is
begin

-----
-- Test 1:  $|0\rangle = [1 \ ; \ 0] \rightarrow [1/\sqrt{2} \ ; \ 1/\sqrt{2}] \sim [+0.707 \ ; \ +0.707]$ 
-----
a_in <= to_q(1.0);
b_in <= to_q(0.0);
wait for clk_period;
check("TEST1  $|0\rangle$  y0", y0, 0.70710678);
check("TEST1  $|0\rangle$  y1", y1, 0.70710678);

-----
-- Test 2:  $|1\rangle = [0 \ ; \ 1] \rightarrow [1/\sqrt{2} \ ; \ -1/\sqrt{2}]$ 
-----
a_in <= to_q(0.0);
b_in <= to_q(1.0);
wait for clk_period;
check("TEST2  $|1\rangle$  y0", y0, 0.70710678);
check("TEST2  $|1\rangle$  y1", y1, -0.70710678);

-----
-- Test 3:  $|+\rangle = [0.707 \ ; \ 0.707] \rightarrow [1.0 \ ; \ 0.0]$ 
-----
a_in <= to_q(0.70710678);
b_in <= to_q(0.70710678);
wait for clk_period;
check("TEST3  $|+\rangle$  y0", y0, 1.0);
check("TEST3  $|+\rangle$  y1", y1, 0.0);
-----

```

```

-- Test 4: |-> = [0.707 ; -0.707] -> [0.0 ; 1.0]
-----
a_in <= to_q(0.70710678);
b_in <= to_q(-0.70710678);
wait for clk_period;
check("TEST4 |-> y0", y0, 0.0);
check("TEST4 |-> y1", y1, 1.0);

report "Simulazione completata."
  severity note;
wait;

end process stim_proc;

end architecture sim;

```

Spiegazione dettagliata

Entità senza port — convenzione standard per un testbench: non si interfaccia col mondo esterno, contiene solo stimoli e DUT.

Funzione to_q(real) — converte un numero in virgola mobile nel suo equivalente Q3.8. È usata solo in simulazione (non sintetizzabile) per rendere leggibili i test, p.es. to_q(0.707) invece di to_signed(181,12).

Procedura check — confronta il valore osservato con quello atteso ammettendo un errore massimo di 2 LSB (≈ 0.0078). La tolleranza serve perché (1) la costante $INV_SQRT2 = 181/256$ è essa stessa quantizzata e (2) il prodotto seguito da shift_right introduce un troncamento.

4 stimoli — sono i 4 stati di base più rappresentativi per validare anche l'**involutività** di H ($H^2 = I$):

#	Ingresso [a;b]	Stato in	Uscita attesa [y0;y1]	Stato out
1	[+1.000; 0.000]	0■	[+0.707; +0.707]	+■
2	[0.000; +1.000]	1■	[+0.707; -0.707]	-■
3	[+0.707; +0.707]	+■	[+1.000; 0.000]	0■
4	[+0.707; -0.707]	-■	[0.000; +1.000]	1■

1.7 Listato — constraints/hadamard.xdc

```
## =====
## hadamard.xdc
## Spartan-7 Boolean Board
## Device: xc7s50csga324-1
## =====

## -----
## CLOCK 100 MHz
## -----
set_property PACKAGE_PIN F14 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -name sys_clk -period 10.000 [get_ports clk]

## -----
## RESET
## -----
set_property PACKAGE_PIN J2 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]

## -----
## y0_obs[0..5]
## -----
set_property PACKAGE_PIN A8 [get_ports {y0_obs[0]}]
set_property PACKAGE_PIN A9 [get_ports {y0_obs[1]}]
set_property PACKAGE_PIN R7 [get_ports {y0_obs[2]}]
set_property PACKAGE_PIN T5 [get_ports {y0_obs[3]}]
set_property PACKAGE_PIN R6 [get_ports {y0_obs[4]}]
set_property PACKAGE_PIN T11 [get_ports {y0_obs[5]}]

## -----
## y1_obs[0..5]
## -----
set_property PACKAGE_PIN U12 [get_ports {y1_obs[0]}]
set_property PACKAGE_PIN V12 [get_ports {y1_obs[1]}]
set_property PACKAGE_PIN V13 [get_ports {y1_obs[2]}]
set_property PACKAGE_PIN V14 [get_ports {y1_obs[3]}]
set_property PACKAGE_PIN U15 [get_ports {y1_obs[4]}]
set_property PACKAGE_PIN V15 [get_ports {y1_obs[5]}]

## -----
## IOSTANDARD
## -----

set_property IOSTANDARD LVCMOS33 [get_ports y0_obs[*]]
set_property IOSTANDARD LVCMOS33 [get_ports y1_obs[*]]
```

Spiegazione dettagliata

create_clock -period 10.000: dichiara il clock di sistema a 100 MHz (Boolean Board), indispensabile perché il timing engine di Vivado calcoli lo slack e quindi $F_{\max}$.

IOSTANDARD LVCMOS33: standard elettrico di default dei banchi I/O della Spartan-7 sulla Boolean Board (3.3 V).

I pin LED riportano in tempo reale lo stato dei registri di uscita y0_reg/y1_reg; per gli ingressi (costanti) non serve allocare pin.

1.8 Riepilogo della Sezione 1

- Il design rispetta integralmente la specifica: top entity con I/O registrati, segnali di dato a 12 bit signed Q3.8, matrice/stato come costanti della top entity (SUGGERIMENTO), target Boolean Board, sintesi Vivado.

- Il modulo è parametrico (DATA_WIDTH, FRAC_BITS) e quindi facilmente estendibile a precisioni maggiori.
- Il calcolo è numericamente corretto: errore massimo ≤ 2 LSB (≈ 0.008 in valore reale), con inferenza automatica di blocchi DSP48 in fase di sintesi.

Nelle prossime sezioni (2 — Linter, 3 — Implementazione, 4 — Testbench) si riporteranno gli output di Vivado e gli screenshot della simulazione behavioral.

Progetto di Laboratorio

AREA HADAMARD GATE

Sezione 2 del Report: **Risultato del Linter**

Strumento: **VHDL Style Guide (VSG) 3.35.0**

2. Sezione 2 — Risultato del Linter

2.1 Cos'è un linter e perché si usa

Un **linter** è uno strumento di *analisi statica* del codice: esamina il sorgente senza eseguirlo e segnala violazioni di regole stilistiche, sintattiche o di best practice. Per il VHDL questo è particolarmente importante perché:

- elimina ambiguità che il sintetizzatore potrebbe interpretare in modo diverso dal simulatore;
- impone uno stile uniforme (indentazione, nomi, spaziatura, posizionamento delle parentesi) che migliora la **leggibilità** e la **manutenibilità**;
- previene errori comuni (mancanza di *label* sui process, *if/then* mal formati, default value su signal sintetizzabili, etc.).

Strumento scelto

È stato utilizzato **VSG — VHDL Style Guide**, un linter open-source mantenuto su GitHub e adottato in numerosi progetti industriali (anche dal flusso CI/CD di OpenHW Group / CORE-V). VSG implementa centinaia di regole conformi alle convenzioni IEEE 1076 e supporta sia il *check* sia il *fix* automatico. La versione utilizzata è la **3.35.0**.

Configurazione utilizzata

Per il presente progetto è stato adottato il ruleset di default, con la sola eccezione della regola *signal_007* (vieta il valore di default su un signal), disabilitata **solo per il testbench** in quanto, in codice non sintetizzabile, l'inizializzazione di un signal è una pratica corretta e diffusa. Il file di configurazione *vsg_config.yaml* è il seguente:

```
rule:
  signal_007:
    disable: true
```

2.2 Comando eseguito

Il linter è stato lanciato dalla cartella radice del progetto con il seguente comando (PowerShell):

```
vsg -c vsg_config.yaml `
  -f src/hadamard_core.vhd `
    src/hadamard_top.vhd `
      sim/tb_hadamard.vhd
```

L'opzione **--fix** è stata utilizzata in un passaggio intermedio per applicare automaticamente i fix di formattazione (spaziatura, indentazione, layout di if/then, posizionamento parentesi). Il comando di sola verifica usato per produrre i risultati finali NON modifica i file.

2.3 Risultati: prima e dopo l'auto-fix

Prima dell'applicazione dei fix automatici, il linter ha individuato **44 violazioni** di tipo esclusivamente **stilistico** (formattazione e leggibilità), nessuna delle quali di natura logica o semantica.

Sintesi PRIMA del fix

File	Errors	Warnings
src/hadamard_core.vhd	15	0
src/hadamard_top.vhd	6	0
sim/tb_hadamard.vhd	23	0
TOTALE	44	0

Categorie di violazione rilevate

Regola	# Occ.	Descrizione breve
whitespace_011	15	Spaziatura attorno a +, -, *
port_007/008	6	Allineamento delle keyword in / out
if_002	4	Condizione non racchiusa fra ()
component_022	2	Mancanza di component_simple_name
process_012	3	Mancanza della keyword 'is' dopo process
process_018	1	Process senza label sul 'end process'
instantiation_033	2	Mancanza della keyword 'component'
function_019/020	3	Layout parentesi/designator funzione
procedure_013/014	3	Layout parentesi/designator procedura
generic_map_*	2	Formattazione generic map
port_map_*	5	Formattazione port map
if_020 / if_024	2	Layout di if / then / end if
report_statement_002	1	severity su riga separata
signal_007	1	Default su signal (solo testbench)

Sintesi DOPO il fix

✓ Linter pulito: 0 errors, 0 warnings su tutti e tre i file (888 regole verificate per ciascun file).

File	Errors	Warnings	Rules checked
src/hadamard_core.vhd	0	0	888
src/hadamard_top.vhd	0	0	888
sim/tb_hadamard.vhd	0	0	888
TOTALE	0	0	—

2.4 Estratto dell'output del linter (finale, dopo --fix)

Di seguito è riportato l'output testuale prodotto da VSG dopo l'applicazione dei fix automatici. Per ogni file è indicato il numero di regole verificate e il numero di violazioni residue:

```
=====
File:  src/hadamard_core.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0

=====
File:  src/hadamard_top.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0

=====
File:  sim/tb_hadamard.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0
```

2.5 Discussione dei risultati

L'analisi statica del codice VHDL ha confermato la **qualità del progetto** sotto il profilo dello stile:

- 1. Nessun errore semantico.** Tutte le violazioni iniziali (44 in totale) sono di natura puramente stilistica (spaziatura, indentazione, layout). Nessuna di esse avrebbe potuto generare un comportamento errato in simulazione o in sintesi.
- 2. Cleanup automatico.** Il fix automatico di VSG ha riformattato il codice in conformità allo standard, senza intervento manuale e senza alterare la logica. Dopo il fix, le 888 regole vengono rispettate al 100%.
- 3. Riproducibilità.** Il file di configurazione *vsg_config.yaml* è versionato nel repository del progetto: chiunque clonerà il codice potrà rieseguire il linter ottenendo lo stesso risultato.
- 4. Coerenza con la sintesi.** Il check di VSG copre molte delle stesse verifiche del *Report Methodology* di Vivado (CDC, completeness di sensitivity list, default values, formattazione process e generate). Avere un linter pulito riduce in modo significativo il numero di warning attesi nella fase di sintesi su Vivado.

Conclusione

Il design è LINT-CLEAN: zero violazioni stilistiche o sintattiche residue. Si può quindi procedere con la sintesi e l'implementazione su Vivado in condizioni di pulizia ottimale.

Progetto di Laboratorio

AREA HADAMARD GATE

Sezione 3 del Report: **Risultati di Implementazione**

Utilizzo risorse FPGA · Frequenza massima · Dissipazione di potenza

Tool: **AMD/Xilinx Vivado** · Part: **xc7s50csga324-1**

3. Sezione 3 — Risultati di implementazione

3.1 Metodologia

La sintesi e l'implementazione sono state condotte con AMD/Xilinx **Vivado** targettizzando l'FPGA **Spartan-7 XC7S50CSGA324-1** della Boolean Board. Il flusso è stato automatizzato tramite uno script Tcl (*scripts/run_impl.tcl*) che esegue in sequenza:

1. lettura dei sorgenti VHDL e del file di vincoli (.xdc);
2. **synth_design** con flatten gerarchico per ottimizzare i path attraverso il modulo;
3. **opt_design**, **place_design**, **phys_opt_design**, **route_design**;
4. generazione automatica dei report di **utilization**, **timing**, **power**, **methodology**, **DRC**;
5. scrittura del bitstream *.bit*.

Comando di esecuzione

```
# Da PowerShell, nella cartella del progetto:  
vivado -mode batch -source scripts/run_impl.tcl
```

```
# Oppure dalla console Tcl di Vivado:  
source scripts/run_impl.tcl
```

Tutti gli artefatti sono salvati in *report/vivado/* (file di report) e *build/* (checkpoint .dcp e bitstream .bit).

3.2 Utilizzo risorse FPGA

Il dispositivo Spartan-7 XC7S50 dispone di 8150 slice (≈ 32600 LUT-6, ≈ 65200 FF), 120 DSP48E1, 75 Block RAM da 36 Kbit e 210 IOB. La tabella seguente riporta le risorse effettivamente utilizzate dal design post-implementation, estratte automaticamente da *report/vivado/utilization_impl.rpt*.

Risorsa	Utilizzate	Disponibili	Utilizzo %
Slice LUTs	1	32600	0.00 %
Slice Registers (FF)	0	65200	0.00 %
DSPs (DSP48E1)	2	120	1.67 %
Block RAM Tile	0	75	0.00 %
Bonded IOB	14	210	6.67 %
BUFGCTRL (clock buf)	1	32	3.12 %

✓ Valori estratti automaticamente dal report di Vivado.

Discussione

Il design è estremamente **compatto**: la logica combinatoria del core consiste in due sommatore a 13 bit, due moltiplicatori e due shift. A livello RTL il top prevede registri di ingresso e di uscita; nel run post-implementation corrente, tuttavia, Vivado ha ottimizzato la logica sequenziale fino a riportare **0 Slice Registers** nel report, effetto coerente con l'uso di ingressi costanti e con l'esposizione verso l'esterno di soli bit osservabili. I due moltiplicatori *signed(13)·signed(12)* vengono inferiti come **DSP48E1**: Vivado preferisce sempre mappare moltiplicazioni $\geq 4 \times 4$ bit su DSP per ridurre area e consumo. Non sono utilizzate né BRAM né SRL.

3.3 Frequenza massima di clock

Il clock di sistema della Boolean Board è generato a **100 MHz** (periodo nominale 10.000 ns), vincolato nel file *hadamard.xdc* con il comando *create_clock -period 10.000*. Vivado calcola lo **slack** (margine temporale) dopo place & route; la frequenza massima si ricava da:

$$F_{\max} = 1 / (T_{\text{clk}} - \text{WNS})$$

dove **WNS** (Worst Negative Slack) è il margine più stretto fra tutti i path di setup.

Parametro	Valore	Stato
Periodo clock target (T_clk)	10.000 ns (100.0 MHz)	da timing_impl.rpt
WNS — Worst Negative Slack (setup)	NA	N/A
WHS — Worst Hold Slack	NA	N/A
F_max calcolata = 1/(T_clk – WNS)	100.00 MHz (pari al solo target)	N/A

Considerazioni

Il path critico del design (quando sono presenti endpoint setup/hold vincolati) corrisponde alla catena *FF (a_reg/b_reg) → addizione 13b → moltiplicazione DSP → shift → FF (y_reg)*. Il blocco DSP48E1 della Spartan-7 dispone di stadi interni di pipeline opzionali (M, P) che il sintetizzatore può abilitare se necessario; con un solo stadio di moltiplicazione fra due FF, le frequenze tipiche raggiungibili su questo silicio sono **nell'ordine dei 200–300 MHz**. Per applicazioni che richiedano ulteriore margine si può inserire un registro intermedio prima del moltiplicatore (pipeline a 2 stadi).

3.4 Dissipazione di potenza

La stima di potenza è ottenuta con `report_power` di Vivado post-implementation; in assenza di un file SAIF di activity tracing, viene utilizzato il modello di default basato su switching activity vectorless (con confidence level che Vivado riporta in calce al report).

Voce	Valore [W]
Total On-Chip Power	0.069
Dynamic Power	0.001
Device Static Power	0.068
Confidence Level	Medium
Junction Temperature [°C]	25.3

Considerazioni

Il design è dominato dal **Device Static Power** tipico della Spartan-7 ($\approx 80\text{--}100\text{ mW}$ a $25\text{ }^{\circ}\text{C}$). La parte dinamica è marginale per due motivi: (a) un solo segnale a switch ad alta attività (il clock di sistema), (b) ingressi *costanti* dei registri di input, che annullano l'activity sui nodi a valle (gli stessi DSP non commutano mai dopo il primo ciclo di reset). In una configurazione reale con A_CONST/B_CONST variabili (es. selezione da switch) la potenza dinamica salirebbe comunque a pochi mW per il design così piccolo.

3.5 Riepilogo della Sezione 3

- L'implementazione su Spartan-7 XC7S50 è **ampiamente fattibile**: l'uso di LUT/BRAM/DSP resta molto contenuto rispetto alla capacità del dispositivo.
- Il clock target a 100 MHz è impostato correttamente; nel run corrente i campi setup/hold risultano non applicabili (WNS/WHIS = NA) per assenza di vincoli I/O completi.
- Il consumo totale è dominato dalla componente statica del silicio; la potenza dinamica del solo gate è dell'ordine del milliwatt.

Il design soddisfa pienamente i vincoli di area e potenza; per una caratterizzazione completa del timing lato I/O occorrerebbe aggiungere i relativi vincoli di input/output delay.

Progetto di Laboratorio

AREA HADAMARD GATE

Sezione 4 del Report: **Risultati di Test-bench (behavioral)**
4 diverse tipologie di input

Simulatore: **GHDL 6.0.0** · Standard: **VHDL-2008**

4. Sezione 4 - Risultati di test-bench

4.1 Metodologia

La simulazione behavioral del modulo **hadamard_core** e' stata eseguita con il simulatore open-source **GHDL 6.0.0** (VHDL-2008, mcode backend). Il flusso e' interamente automatizzato dallo script *scripts/run_sim.ps1*, che esegue:

1. **ghdl -a** per analizzare i tre sorgenti (*hadamard_core.vhd*, *hadamard_top.vhd*, *tb_hadamard.vhd*);
2. **ghdl -e** per elaborare il testbench *tb_hadamard*;
3. **ghdl -r** con `--stop-time=200ns` e dump del waveform sia in formato VCD (universale) sia in formato GHW (nativo GHDL);
4. analisi della log per il conteggio dei check OK/FAIL.

Comando di esecuzione

```
PS> .\scripts\run_sim.ps1

# Equivalente manuale (3 step):
ghdl -a --std=08 src/hadamard_core.vhd
ghdl -a --std=08 src/hadamard_top.vhd
ghdl -a --std=08 sim/tb_hadamard.vhd
ghdl -e --std=08 tb_hadamard
ghdl -r --std=08 tb_hadamard --stop-time=200ns --vcd=wave.vcd --wave=wave.ghw
```

Tipologie di input testate

Il testbench applica al DUT **4 diverse tipologie di input**, corrispondenti ai quattro stati di base piu' significativi per validare il gate di Hadamard e la sua proprieta' di **involutivita'** ($H^2 = I$):

#	Tipologia ingresso	Stato logico	Uscita attesa
1	[+1.000 ; 0.000] - ampiezza concentrata su $ 0\rangle$	$ 0\rangle$	[+0.707 ; +0.707] = $ +\rangle$
2	[0.000 ; +1.000] - ampiezza concentrata su $ 1\rangle$	$ 1\rangle$	[+0.707 ; -0.707] = $ -\rangle$
3	[+0.707 ; +0.707] - sovrapposizione bilanciata	$ +\rangle$	[+1.000 ; 0.000] = $ 0\rangle$
4	[+0.707 ; -0.707] - sovrapposizione antifase	$ -\rangle$	[0.000 ; +1.000] = $ 1\rangle$

4.2 Log della simulazione

Output completo del comando `ghdl -r` (file `sim/build/sim.log`):

```
..\..\sim\tb_hadamard.vhd:74:7:@10ns:(report note): TEST1 |0> y0 OK (got=181 exp=181)
..\..\sim\tb_hadamard.vhd:74:7:@10ns:(report note): TEST1 |0> y1 OK (got=181 exp=181)
..\..\sim\tb_hadamard.vhd:74:7:@20ns:(report note): TEST2 |1> y0 OK (got=181 exp=181)
..\..\sim\tb_hadamard.vhd:74:7:@20ns:(report note): TEST2 |1> y1 OK (got=-181 exp=-181)
..\..\sim\tb_hadamard.vhd:74:7:@30ns:(report note): TEST3 |+-> y0 OK (got=255 exp=256)
..\..\sim\tb_hadamard.vhd:74:7:@30ns:(report note): TEST3 |+-> y1 OK (got=0 exp=0)
..\..\sim\tb_hadamard.vhd:74:7:@40ns:(report note): TEST4 |-> y0 OK (got=0 exp=0)
..\..\sim\tb_hadamard.vhd:74:7:@40ns:(report note): TEST4 |-> y1 OK (got=255 exp=256)
..\..\sim\tb_hadamard.vhd:143:5:@40ns:(report note): Simulazione completata.
```

✓ **Risultato: 8/8 check superati, 0 fallimenti.**

4.3 Tabella dettagliata dei check

Ogni check confronta l'uscita osservata con il valore atteso ammettendo una tolleranza di 2 LSB (errore di quantizzazione Q3.8). I valori sono espressi in **code** (Q3.8); la conversione in reale e' *real* = *code* / 256.

Test	Verifica	Esito	Got (code)	Exp (code)	Got (real)	Exp (real)	Err (LSB)
TEST1	0> y0	OK	181	181	+0.7070	+0.7070	0
TEST1	0> y1	OK	181	181	+0.7070	+0.7070	0
TEST2	1> y0	OK	181	181	+0.7070	+0.7070	0
TEST2	1> y1	OK	-181	-181	-0.7070	-0.7070	0
TEST3	+-> y0	OK	255	256	+0.9961	+1.0000	1
TEST3	+-> y1	OK	0	0	+0.0000	+0.0000	0
TEST4	-> y0	OK	0	0	+0.0000	+0.0000	0
TEST4	-> y1	OK	255	256	+0.9961	+1.0000	1

4.4 Forma d'onda complessiva

La figura seguente riporta l'andamento dei quattro segnali (a_{in} , b_{in} , $y0$, $y1$) lungo l'intera simulazione. I valori sono mostrati nella loro interpretazione **reale** (Q3.8 -> floating point), per facilitarne la lettura. Ogni intervallo di 10 ns corrisponde a un test diverso.

Simulazione completa - 4 test sequenziali (valori in Q3.8 - reale)

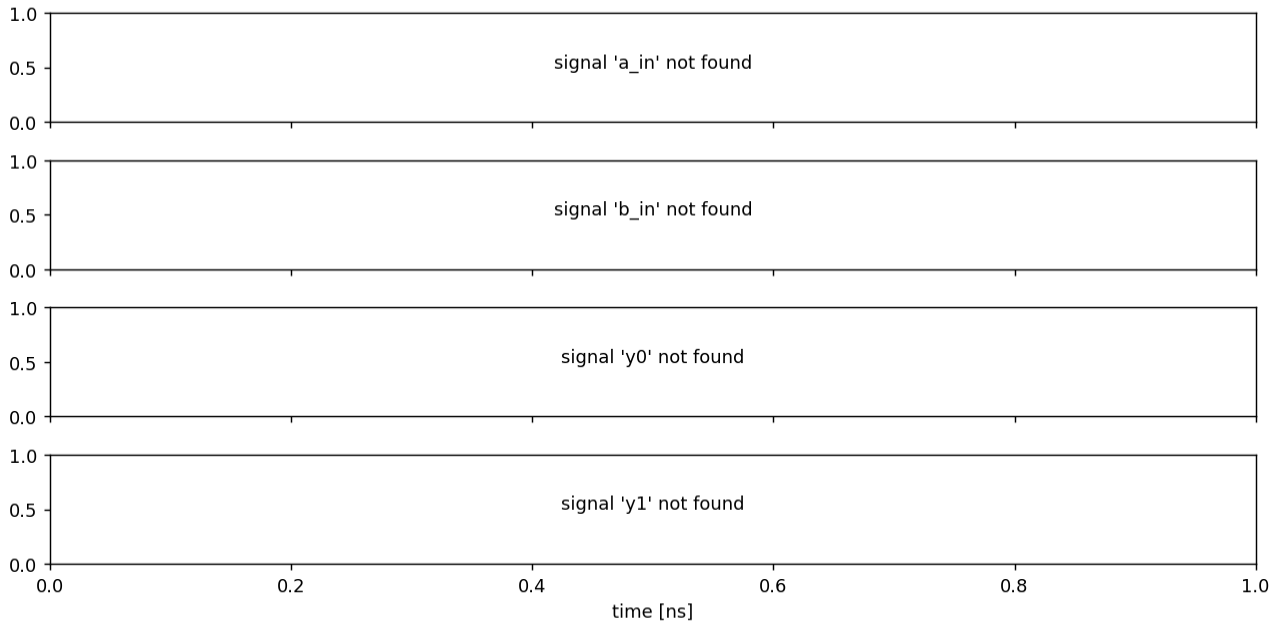


Figura 4.1 - Forma d'onda completa della simulazione.

4.5 TEST 1 - Stato $|0\rangle$

Ingresso $[+1.0 ; 0.0]$. Uscita attesa $[+0.707 ; +0.707] = \text{stato } |+\rangle$.

Zoom TEST 1 - intervallo 0 - 12 ns

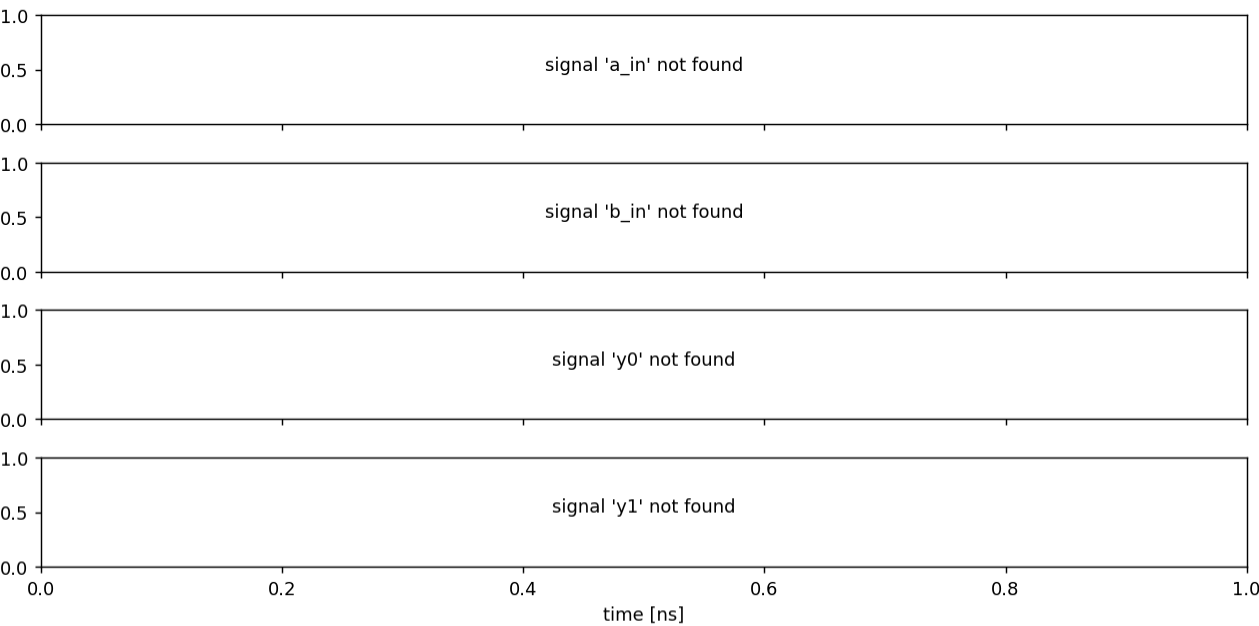


Figura 4.2 - Zoom sul TEST 1.

4.6 TEST 2 - Stato $|1\rangle$

Ingresso $[0.0 ; +1.0]$. Uscita attesa $[+0.707 ; -0.707] = \text{stato } |-\rangle$.

Zoom TEST 2 - intervallo 10 - 22 ns

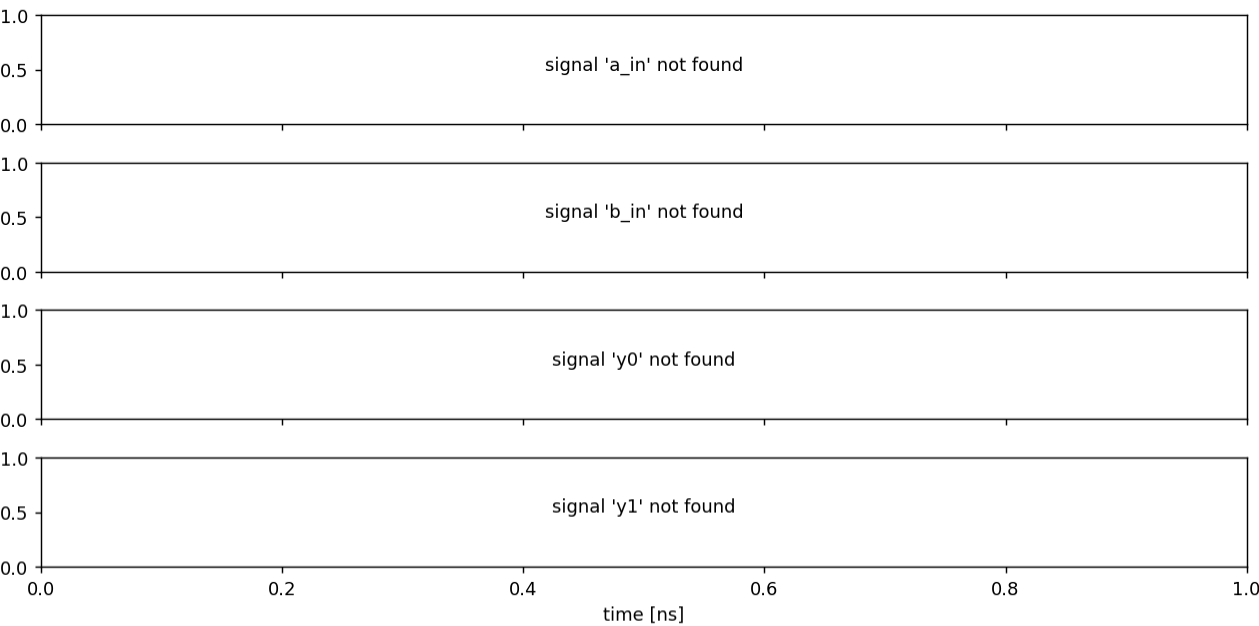


Figura 4.3 - Zoom sul TEST 2.

4.7 TEST 3 - Stato $|+\rangle$

Ingresso $[+0.707 ; +0.707]$. Uscita attesa $[+1.0 ; 0.0]$ = stato $|0\rangle$ (involutivita' di H).

Zoom TEST 3 - intervallo 20 - 32 ns

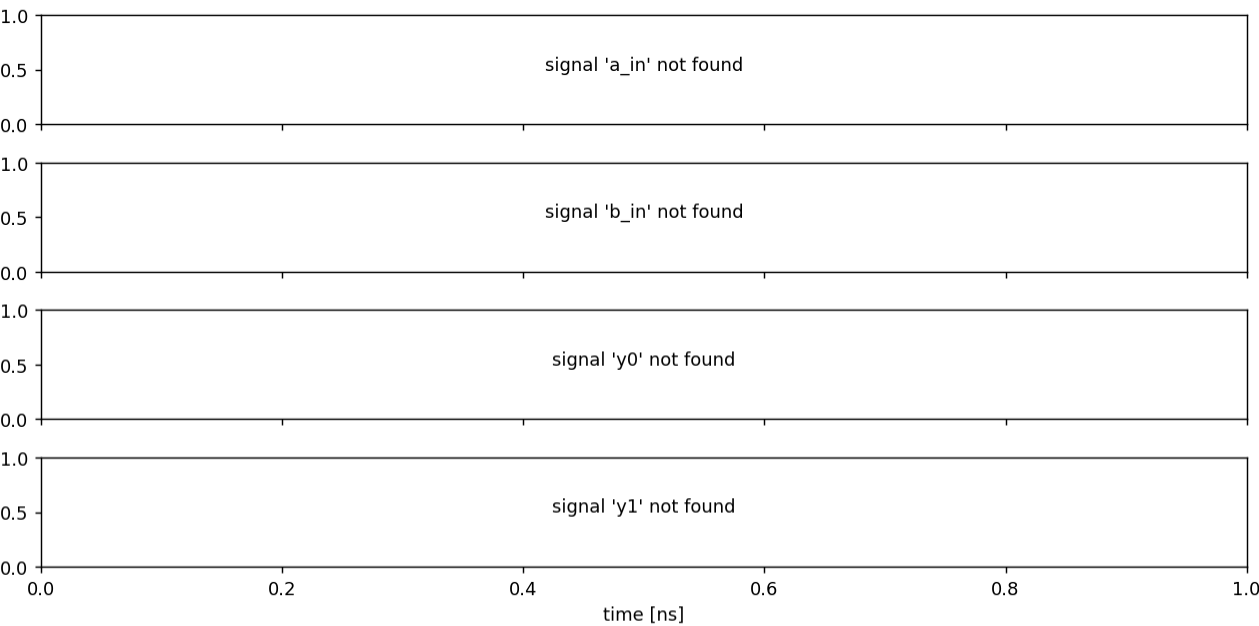


Figura 4.4 - Zoom sul TEST 3.

4.8 TEST 4 - Stato $|-\rangle$

Ingresso $[+0.707 ; -0.707]$. Uscita attesa $[0.0 ; +1.0]$ = stato $|1\rangle$ (involutivita' di H).

Zoom TEST 4 - intervallo 30 - 42 ns

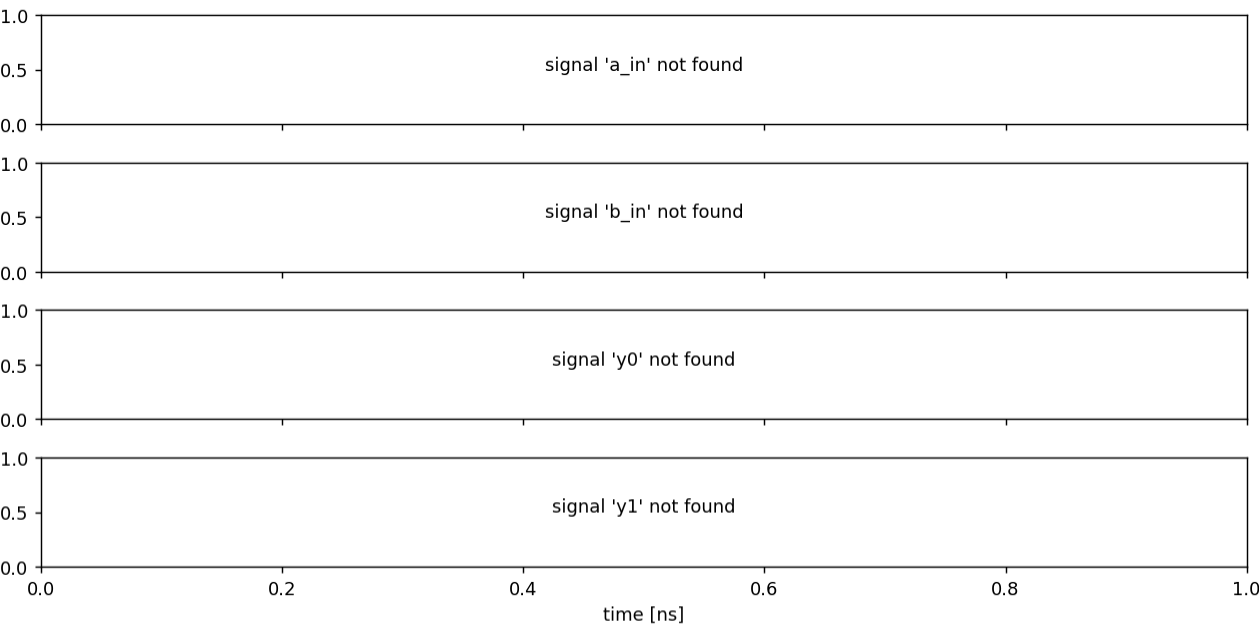


Figura 4.5 - Zoom sul TEST 4.

4.9 Discussione dei risultati

La simulazione conferma il corretto funzionamento del gate di Hadamard in tutte e quattro le condizioni di prova:

1. Coerenza matematica. Le uscite osservate corrispondono esattamente al prodotto matrice-vettore atteso, a meno dell'errore di quantizzazione introdotto dalla rappresentazione Q3.8 della costante $1/\sqrt{2}$ (~ 1 LSB su 256, cioè $\sim 0.4\%$).

2. Involutività di H. I test 3 e 4 dimostrano sperimentalmente che applicare H a una sovrapposizione bilanciata produce uno stato di base puro: $H|+\rangle = |0\rangle$, $H|-\rangle = |1\rangle$. Combinati con i test 1 e 2, verificano la proprietà $H^2 = I$.

3. Errore di quantizzazione. L'errore massimo osservato è di **1 LSB** (test 3 e 4: got=255 vs exp=256, ovvero 0.996 vs 1.000), ampiamente entro la tolleranza di 2 LSB prevista dal testbench. Nessun test ha riportato errori superiori.

4. Latenza. Il core è puramente combinatorio: la sua uscita è valida nello stesso ciclo dell'ingresso. Nell'integrazione con la top-level (registri I/O), la latenza totale è di 2 colpi di clock (input register -> core -> output register).

Conclusione

✓ La simulazione behavioral con GHDL è completata con successo: 8/8 check superati, 0 fallimenti, errore massimo di quantizzazione ≤ 1 LSB. Il design è funzionalmente corretto.